

One LangGraph agent, wired to Langfuse in one run and to OpenBox in the other.

The runs used the same agent, prompt, documents and model, and did the same work: 3 searches, 3 document reads, 1 briefing, 4 filings (11 tool calls). Below is what each platform recorded and what its dashboard shows. The two overlap on the agent-level trace. Langfuse goes deeper into the framework. OpenBox goes deeper into the running process, and records a decision and a signed audit trail on top.

THE SETUP

Agent	Run 1 · LangGraph + Langfuse	Run 2 · LangGraph + OpenBox	No policies
Amy, a deal analyst agent from our Policy Studio demo. LangGraph, 4 tools: search_documents, read_document, write_briefing, upload_document.	Langfuse's LangChain CallbackHandler passed in the graph config. No OpenBox.	The graph wrapped with create_openbox_graph_handler. No Langfuse.	Amy has no rules, so nothing was blocked and both runs did identical work.

What each one recorded

Langfuse · 56 observations	OpenBox · 27 events + 14 spans
43 graph steps: every LangGraph node and router (schedule_search, select_candidate, after_read ...) 11 tool calls, each with input and full output 1 LLM generation: model, temperature, messages, 633 in / 288 out tokens 1 root agent span	27 governance events: prompt, session start and end, a start and an end for each of the 12 tool and LLM calls, each with a verdict 13 file operations: 8 reads, 5 writes, each with path, mode and bytes 1 HTTP call to the model: URL, headers, request and response bodies, status 1 signed session record: Merkle root over 55 leaves (the 27 events, plus each of the 14 spans signed once when it starts and once when it completes), 27-link hash chain, RSA-SHA256

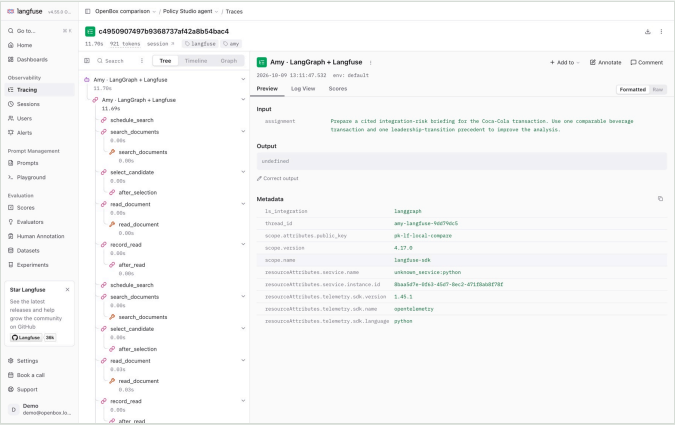
Field by field

DATA	LANGGRAPH + LANGFUSE	LANGGRAPH + OPENBOX
Session and run timeline	Trace, session, tree and timeline views	Session list, replay, execution tree, timeline

DATA	LANGGRAPH + LANGFUSE	LANGGRAPH + OPENBOX
Tool calls and their inputs	Yes	Yes
Latency per step	Yes, per node, tool and LLM call	Yes, per tool and LLM call, and per span
Graph internals: routers and non-tool nodes	Yes, all 43, plus an auto-drawn graph of the agent	Not recorded. Only tool and LLM activities
Tool output	Full return value, e.g. the whole document text	Not on the event. The file span shows path and bytes read, not content
LLM call	Parsed: system and user messages, reply, model, temperature, token counts, cost (when the model has a price)	Raw HTTP: request and response bodies, 17 request headers (auth redacted), status code, duration
Files the agent touched	Not visible	Every read and write with real path, mode and bytes. Includes reads the tool code never asked for (Python read SystemVersion.plist)
Outbound network calls	Only LLM calls, via the callback	Every HTTP request the process makes, with URL, headers and bodies
Decision per call	—	Verdict on every event and every span (all ALLOW here)
Agent identity	A free-text trace name	A registered agent with its own key, trust tier and permissions
Tamper-evidence	—	Per-event Merkle proof, hash chain and signature, checked in the Verify tab
Dashboards	Traces, cost by model, latency percentiles, observation levels	Invocations, tokens, cost, latency P50–P99, tool health, errors by cause
Prompt management, evals, datasets, playground	Yes	—

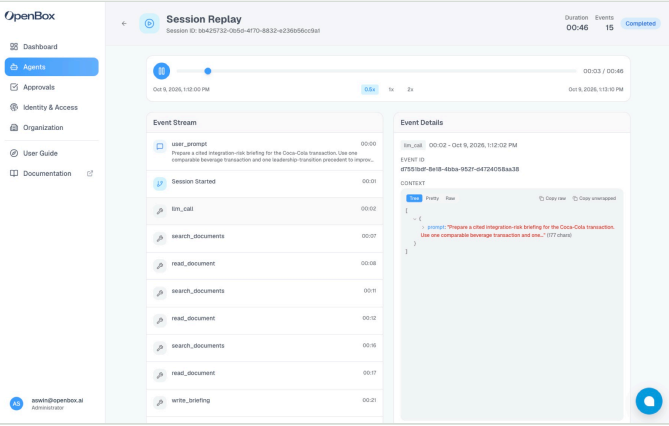
The agent-level trace

LANGFUSE



Trace tree. Every LangGraph node is its own span, including control-flow steps like `schedule_search` and `after_selection`.

OPENBOX

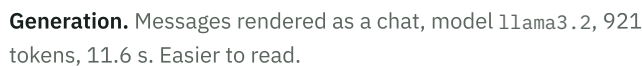


Session replay. Tool and LLM calls in order, with a playhead. Routing nodes don't appear.

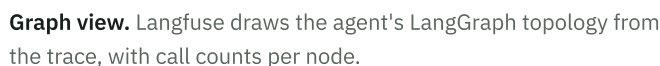
LANGFUSE



LANGFUSE



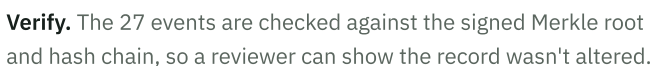
LANGFUSE



Event plus span. The `document_id` in, then the actual file read underneath it: `real path`, `mode` and its own `ALLOW` verdict.

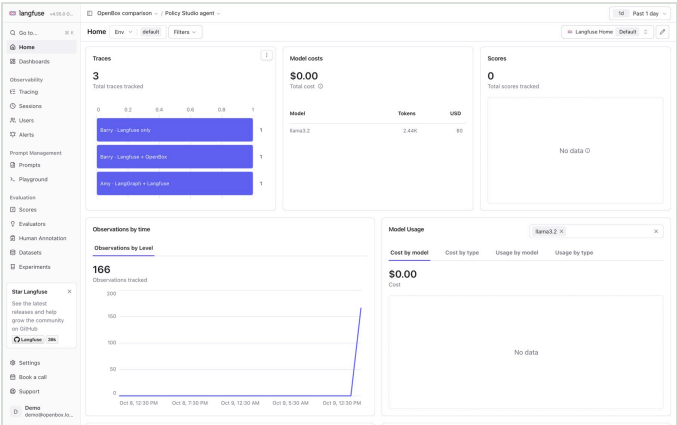
HTTP span. The request as it left the process: endpoint, method, headers, request and response bodies, status. Closer to the wire.

OPENBOX



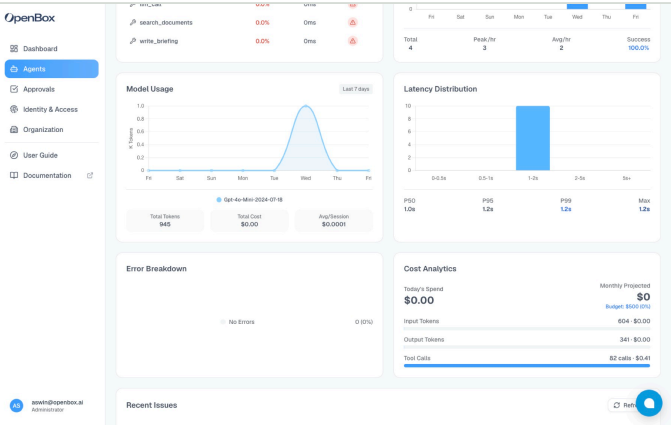
Dashboards

LANGFUSE



Project home. Trace counts, model cost and tokens, observations over time.

OPENBOX



Agent overview. Token usage, cost, latency distribution and errors by cause, per agent. These are 7-day figures. The token and cost figures come from an earlier gpt-4o-mini run of Amy, because the panel didn't count the local model used in the compared run.

Overlap, and where it stops

Both

- Hook into LangGraph with a few lines
- Session timeline of tool and LLM calls, with inputs and latency
- Per-agent or per-project usage dashboards

Langfuse goes deeper into the framework

- Every graph node and router, and an auto-drawn graph
- Tool outputs
- LLM calls parsed into messages, tokens and cost
- Prompt management, evals, datasets

OpenBox goes deeper into the process

- Real file reads and writes, and outbound HTTP, below the tool
- A verdict on every event and span
- A registered identity for each agent
- A signed, verifiable record of the session

The overlap is the agent-level trace, which any LangGraph integration can produce. Langfuse is built to debug and improve the agent. OpenBox records what the agent actually did on the system, in a form an auditor can verify, and the same hooks enforce policy.

METHOD AND LIMITS

Both runs were made on 9 Oct 2026, against a local OpenBox stack (develop) and a self-hosted Langfuse v4.55 (Python SDK 4.17). Counts come from each platform's own store: Langfuse's ClickHouse, and OpenBox's governance events and spans tables. The model was a local llama3.2, so Langfuse shows no cost. OpenBox's token panel didn't count this local model, so its token figures come from the earlier gpt-4o-mini run. The documents are fictional demo files; the company names in them (Coca-Cola, Dell, Staples) are real brands used as placeholders, and nothing in them comes from those companies.

Reproduce: `uv run --project ../openbox-barrier-demo --with langfuse --with langchain python run_compare.py --mode langfuse, then --mode openbox.`